

Divide and Conquer – Summary Objects and $O(1)$ “combine” step

Motivation

Consider the following problem:

Longest Increasing Contiguous Subarray

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: The *length* of the Longest Increasing Contiguous Subarray (LICS).

Example:

Given: 1, 2, 3, 7, 5, 6, 7, 8, 9, 2, 4, 5, 6, 8, 9, 1, 2

Motivation

Consider the following problem:

Longest Increasing Contiguous Subarray

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: The *length* of the Longest Increasing Contiguous Subarray (LICS).

Example:

Given: 1, 2, 3, 7, 5, 6, 7, 8, 9, **2, 4, 5, 6, 8, 9**, 1, 2

Return: 6 (corresponding to the subarray 2, 4, 5, 6, 8, 9)

Can you solve this in $O(n)$ by scanning left to right?

Motivation

Consider the following problem:

Longest Increasing Contiguous Subarray

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: The *length* of the Longest Increasing Contiguous Subarray (LICS).

Example:

Given: 1, 2, 3, 7, 5, 6, 7, 8, 9, **2, 4, 5, 6, 8, 9**, 1, 2

Return: 6 (corresponding to the subarray 2, 4, 5, 6, 8, 9)

Can you solve it in $O(n)$ using a Divide and Conquer strategy?

- A tempting idea is “consider each half of the array. Find the LICS in either half, return the length of the longest of these two”.
- Can you find an input where this fails?
- What possibility is the suggestion ahead not accounting for?

Motivation

Consider the following problem:

Longest Increasing Contiguous Subarray

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: The *length* of the Longest Increasing Contiguous Subarray (LICS).

Example:

Given: 1, 2, 3, 7, 5, 6, 7, 8, 9, **2, 4, 5, 6, 8, 9**, 1, 2

Return: 6 (corresponding to the subarray 2, 4, 5, 6, 8, 9)

We need to account for the possibility that the LICS isn't strictly contained in either half, but that it may cross over

- Simple fix: linear scan from the middle
- What is the recurrence relationship characterizing this solution? What is its time complexity?

Summary Objects in D&C

For most algorithms we've seen, each recursive instance only needs to return the answer to the problem. But this is not a requirement.

- Idea: each instance can also return information that helps its parent reconstruct the solution of the parent problem.
- What if we knew:
 - The length of the LICS in each child instance and
 - The length of the longest *prefix* in each child instance
 - The length of the longest *suffix* in each child instance
- Then our solution becomes the longest of:
 - The length of the LICS in the left
 - The length of the LICS in the right
 - The length of the left *suffix* plus the length of the right *prefix*
 - But only if we can “glue”

Longest Increasing Contiguous Subarray

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: The *length* of the Longest Increasing Contiguous Subarray (LICS).

Consider the following code. What is it missing?

$LICS(A[1, \dots, n], lo, hi)$

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$ indices lo (initially $lo = 1$), and hi (initially, $hi = n$)

Goal: Return a summary object containing:

- a) The length of the LICS in $A[lo..hi]$
 - b) The length of the longest increasing *prefix* in $A[lo..hi]$
 - c) The length of the longest increasing *suffix* in $A[lo..hi]$
1. $mid \leftarrow \lfloor \frac{(lo+hi)}{2} \rfloor$
 2. $Summary_L \leftarrow LICS(A, lo, mid)$
 3. $Summary_R \leftarrow LICS(A, mid + 1, hi)$
 4. $Answ \leftarrow new\ Summary(LICS \leftarrow -\infty, Prefix \leftarrow -\infty, Suffix \leftarrow -\infty)$
 5. $Answ.LICS \leftarrow \max(Summary_L.LICS, Summary_R.LICS)$
 6. $canMerge \leftarrow A[mid] < A[mid + 1]$
 7. *if* $canMerge$: $Answ.LICS \leftarrow \max(Answ.LICS, Summary_L.Suffix + Summary_R.Prefix)$
-

Longest Increasing Contiguous Subarray

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: The *length* of the Longest Increasing Contiguous Subarray (LICS).

We need:

1. A base case
 1. This should be easy. If $lo = hi$, then the length of the LICS, Prefix and Suffix are all =1
2. A way to calculate the new prefix and suffix
 1. Is it true that the suffix of $A[lo..hi]$ is always the suffix of the right half (and conversely for the prefix and left)?
 2. You can check whether the prefix or suffix “rolls over” to the other half if
 1. The prefix or suffix already spans the entire range of either half and
 2. The *canMerge* condition is met

Longest Increasing Contiguous Subarray

Input: A list of numbers, $a_1, a_2, a_3, \dots, a_n$.

Goal: The *length* of the Longest Increasing Contiguous Subarray (LICS).

We need:

1. A base case
 1. This should be easy. If $lo = hi$, then the length of the LICS, Prefix and Suffix are all =1
2. A way to calculate the new prefix and suffix
 1. Is it true that the suffix of $A[lo..hi]$ is always the suffix of the right half (and conversely for the prefix and left)?
 2. You can check whether the prefix or suffix “rolls over” to the other half if
 1. The prefix or suffix already spans the entire range of either half and
 2. The *canMerge* condition is met

Recap of Summaries

An algorithm may need more information than the answer to each subproblem in order to solve its own instance. In these cases we need:

- To identify what other “helper” information we need
 - In this case Prefix and Suffix length
- To explain how to solve the original problem given these “helpers”
 - This is typically the best of
 - The left solution
 - The right solution
 - Something that uses the “helper” to identify if an even better solution exists that uses both halves
- To be able to update the helper information in order to return it to the parent instance
- To define the values of the helpers in the base case

Recap of Summaries

Using summaries in this way lets us do $O(1)$ “merge” or “combine” steps to solve several problems like:

- Several variations of LICS
 - Contiguous subarray with highest sum (not necessary increasing) **problem 9 in the homework sheet**
 - Contiguous increasing array with highest sum
 - Contiguous subarray of only prime numbers
- Given a list of daily stock price, the (buy, sell) date pair with $buy \leq sell$ that gives the highest profit **problem 10 in the homework sheet**

All of these problems can be done in $O(n)$ with a left-to-right scan (or dynamic programming approach), but careless divide and conquer could cost $O(n * \log n)$ without keeping track of the proper helper information

Recap of Summaries

Another way to think about this is that any aggregate information (min, max, average, sum, count of elements with a specific property....) that can be computed of an array in $O(n)$ can typically be included “for free” as part of a summary in a D&C algorithm, without affecting time or space complexity